Apellidos: ..... Nombre: ..... Grup: .....

Examen final IC (*Parte 2)*

- Duración del examen: 2:30 horas.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución del examen se publicará en Atenea mañana por la mañana.

Nota: La indicación de la puntuación de los ejercicios es sobre 10 puntos, pero esta parte del examen final solo representa 5 puntos de la nota del examen final.

Ejercicio 1 *(1.9 puntos)*

El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina para ser ejecutado en el SISC Von Neumann, situando la sección de datos (`.data`) a partir de la dirección **`0x3000`** y la sección de código (`.text`) a partir de la dirección **`0x3500`**.

```
.data
        long=9
        mul=2
        A: .byte 4, 12, 6, -8, 0, -2, 3, 3, -1
           .even
        B: .space 4
.text
        MOVI  R0, long
        MOVI  R1, lo(A)
        MOVHI R1, hi(A)
        ADDI  R2, R1, long-1
        MOVI  R3, mul
        MOVI  R7, 0
loop:   LDB   R4, 0(R1)
        LDB   R5, 0(R2)
        SHA   R6, R5, R3
        SUB   R6, R4, R6
        BNZ   R6, 1
        ADDI  R7, R7, 1
        ADDI  R1, R1, 1
        ADDI  R2, R2, -1
        ADDI  R0, R0, -1
        BNZ   R0, loop
cont:   MOVI  R1, lo(B)
        MOVHI R1, hi(B)
        ST    2(R1), R7
.end
```

a) Una vez ensamblado y cargado el programa en la memoria, ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.4 puntos)*

B= **0x** _____ **loop=** **0x** _____

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@ memoria y contenido
`SHA R6, R5, R3`	Mem_w[0x] = 0x
`BNZ R0, loop`	Mem_w[0x] = 0x
`LDB R5, 0(R2)`	Mem_w[0x] = 0x

c) Una vez ejecutado el programa en el SISC von Neumann, completa la siguiente tabla con el contenido de la memoria en hexadecimal (no es necesario poner 0x del valor). Las posiciones de memoria que no se pueda saber su contenido dejadlas en blanco. *(0.6 puntos)*

@memoria	Valor
0x3000	
0x3001	
0x3002	
0x3003	

@memoria	Valor
0x3004	
0x3005	
0x3006	
0x3007	

@memoria	Valor
0x3008	
0x3009	
0x300A	
0x300B	

@memoria	Valor
0x300C	
0x300D	
0x300E	
0x300F	

d) Si se ejecutase el programa en el computador en la SISC Harvard multiciclo, indicad el número total de instrucciones que ejecutaría, así como cuántas son lentas y cuántas son rápidas. *(0.3 puntos)*

N_{total} = _____ **N_{lentas} =** _____ **$N_{rápidas}$=** _____

Ejercicio 2 *(2 puntos)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UCG en el anexo. Suponed que el contenido de todos los registros, `Rk` para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido del registro IR. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.4 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Valor del IR** (en hexadecimal)
Stb	`STB 1(R1), R2`	0x
Bz	`BZ R2, -3`	0x
Addr	`LDB R1, 4(R2)`	0x
Out	`OUT 11, R7`	0x

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.8 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
Stb	`STB 1(R1), R2`	0x
Bz	`BZ R2, -3`	0x
Addr	`LDB R1, 4(R2)`	0x
Out	`OUT 11, R7`	0x

c) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT). *(0.8 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
		@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	LdIr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
Stb	`STB 1(R1), R2`																			
Bz	`BZ R2, -3`																			
Addr	`LDB R1, 4(R2)`																			
Out	`OUT 11, R7`																			

Ejercicio 3 *(1.2 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x4A58`, el contenido de todos los registros pares es `0x2222` y el de los registros impares es `0x3333`, el byte contenido en todas las direcciones pares de la memoria es `0x89` y el de todas las impares es `0xA5` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x…]=0x... y/o MEM_w[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Apellidos: ..... Nombre: ..... Grup: .....

Instrucción a ejecutar	Cambios en el estado del computador
`STB 5(R2), R4`	
`JALR R7, R0`	
`LD R5, 17(R1)`	

Ejercicio 4 *(1 punto)*

Se ha conectado a la UPG un dispositivo externo de entrada que nos envía valores y que tiene el registro de status en la dirección 4 del espacio de direccionamiento de entrada y el de datos en la 5. También se ha conectado un dispositivo externo de salida que tiene el registro de status en la dirección 6 del espacio de direccionamiento de entrada y el de datos en la dirección 7 del espacio de direccionamiento de salida. Ambos dispositivos tienen un efecto lateral en la lectura/escritura del dato sobre su registro de estado.

Se desea implementar un algoritmo en lenguaje SISA que lea un dato (que se interpreta como un número entero en Ca2) del dispositivo de entrada, le cambie el signo (supondremos que nunca hay overflow) y lo envíe al dispositivo de salida. Este proceso debe realizarse indefinidamente hasta que el dato enviado al dispositivo de salida sea el 0.

Ejercicio 5 *(1.6 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 90 u.t.

T_p(ROM_OUT) = 120 u.t.

T_p(MUX-2-1) = 40 u.t.

T_p(MUX-4-1) = 80 u.t.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 300 u.t. // Tiempo de lectura del banco de registros

T_p(ALU-slow) = 800 u.t. // Tp de la ALU para las operaciones/funciones lentas: ADD, SUB, CMP*.

T_p(ALU-quick) = 500 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de ADD, SUB, CMP*.

T_{acc}(64KB MEMORY) = 900 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

a) T_c correspondiente al nodo de **D** (decode).

T_c=

b) T_c correspondiente al nodo de **Movhi**.

T_c=

c) T_c correspondiente al nodo de **Ld**.

T_c=

d) T_c correspondiente al nodo de **Jalr**.

T_c=

Ejercicio 6 *(2.3 puntos)*

Debido a que es muy frecuente en programas tener que leer un conjunto de valores de un dispositivo de entrada y almacenarlo en un vector, se ha decidido crear una nueva instrucción (**INST+**) que haga la mayor parte de este trabajo. Esta nueva instrucción leerá el dato de un dispositivo de entrada y lo almacenará en la posición de memoria indicada por un registro. Además, actualizará el valor del registro para el almacenaje del siguiente valor.

La nueva instrucción tiene el formato y codificación, la sintaxis ensamblador y la semántica siguientes:

Codificación:	`1111 aaa x nnnnnnnn`
Sintaxis:	`INST+ Ra, N8`
Semántica:	`PC=PC+2; MEM`$_w$`[Ra]=INPUT[N8]; Ra=Ra+2`

Donde `Ra` es la dirección de memoria donde se almacenará el primer elemento del vector y `N8` el puerto de datos del dispositivo. Por ejemplo, supongamos que deseamos leer 100 valores de un dispositivo, que tiene el registro de status en el puerto 5 y el de datos en el 6, y almacenarlos en el vector A de words. El código SISA original y con la nueva instrucción para realizar la tarea serían los siguientes:

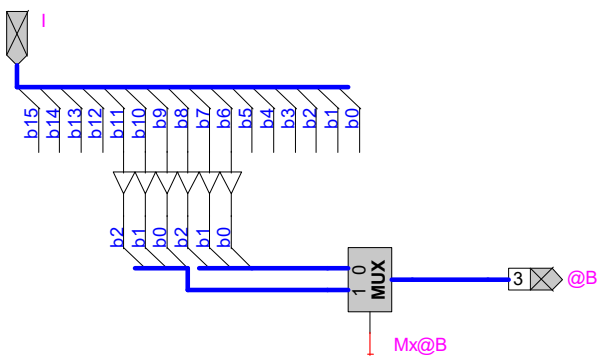
Código original

```
        MOVI  R0, 100
        MOVI  R1, lo(A)
        MOVHI R1, hi(A)
bucle:  IN    R7, 5
        BZ    R7, bucle
        IN    R7, 6
        ST    0(R1), R7
        ADDI  R1, R1, 2
        ADDI  R0, R0, -1
        BNZ   R0, bucle
```

Código con la nueva instrucción

```
        MOVI  R0, 100
        MOVI  R1, lo(A)
        MOVHI R1, hi(A)
bucle:  IN    R7, 5
        BZ    R7, bucle
        INST+ R1, 6
        ADDI  R0, R0, -1
        BNZ   R0, bucle
```

Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, la nueva instrucción **INST+**. Para añadir esta nueva instrucción no es necesario modificar el hardware de la UPG. Para ejecutar esta nueva instrucción solamente se ha modificado el hardware de la unidad de control del computador (UCG) añadiendo un nuevo MUX-2-1 con señal de selección `Mx@B`. La siguiente figura muestra la parte de la Unidad de Control que ha cambiado respecto de la original.

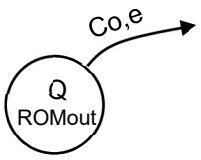


Apellidos: ..... Nombre: ..... Grup: .....

Se pide:

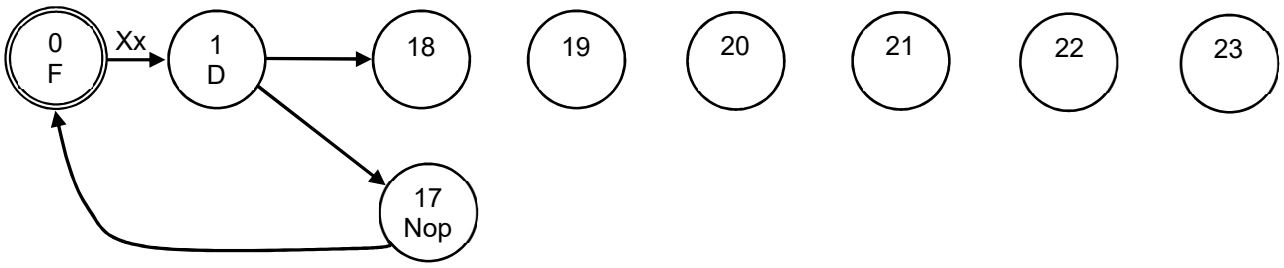
a) Completad el fragmento del grafo de estados de la figura correspondiente al circuito secuencial de la unidad de control. Se da la leyenda del grafo y los nodos necesarios para ejecutar la nueva instrucción, pero faltan arcos y etiquetas. Dibujad todos los arcos que faltan y todas las etiquetas. En número de nodos necesarios para ejecutar la nueva instrucción dependerá de la solución que propongáis, pero no serán más de 6 y deberá ser coherente con las acciones que pongáis en el apartado b). Los nodos que os sobren podéis dejarlos sin conectar. *(0.1 puntos)*

Leyenda:



CO: Código de operación de la Instrucción, I_{15} I_{14} I_{13} I_{12} (en hexadecimal)
e: Bit de extensión del código de operación (I_8)
Q: Estado (en decimal)
ROMout: Mnemotécnico de salida

Grafo:



b) Completad el contenido de la siguiente tabla que indica, mediante una fila para cada nodo del grafo de estados de la unidad de control, la acción (o acciones en paralelo) que se realiza en el computador en cada uno de los ciclos/nodos que requiere la ejecución de la nueva instrucción (Fetch, Decode, y los ciclos/nodos de la ejecución propiamente dicha). Para especificar las acciones usad el mismo lenguaje de transferencia de registros que en la documentación. En número de filas/nodos de la tabla a rellenar dependerá de la solución que propongáis y deberá ser coherente con el apartado a) y c). Nota: Existe una solución solo con 6 nodos (F, D y 4 nodos nuevos). *(0.8 puntos)*

Nodo	Mnemotécnico	Acciones
E0	F	IR ← Mem_w[PC] // PC ← PC+2
E1	D	RX ← Ra // RY ← Rb // R@ ← PC+SE(N8)*2
E18		
E19		
E20		
E21		
E22		
E23		
E24		

Si la solución propuesta en el apartado b) no permite ejecutar correctamente la nueva instrucción, no se corregirán los apartados a) ni c).

c) Completad (poniendo 0, 1 o x en cada bit) la siguiente tabla que especifica, mediante una fila por cada nuevo nodo añadido al grafo de estados de la unidad de control, la dirección (en decimal) de la ROM_OUT y su contenido para que se ejecute correctamente la nueva instrucción. **Poned x siempre que el valor de un bit no importe**. En la columna de la derecha poned el mnemotécnico que habéis usado para nombrar cada nuevo nodo en el grafo del apartado b). *(0.8 puntos)*

@ROM	**Mx@B**	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A_1 P/I/L/A_0	OP_1 OP_0	MxN_1 MxN_0	MxF	F_2 F_1 F_0	Mx@D_1 Mx@D_0	**Nodo (Mnemo)**
18																				
19																				
20																				
21																				
22																				
23																				
24																				

d) Completad la dirección o contenido, según corresponda, de la ROM_Q+ del SISC Von Neumann. *(0.2 puntos)*

`ROM_Q+[0x0A8]=` `0x` ____ `ROM_Q+[` `0x` ____ `]=0x0D`

e) Suponiendo que la ejecución de la nueva instrucción se realiza en 6 ciclos (F, D y 4 nodos nuevos), y asumiendo que siempre hay datos disponibles (registro de status a 1). Indica cuantas instrucciones se ejecutarían y cuantos ciclos se tardaría en ejecutar el programa de ejemplo del inicio del enunciado si ejecutásemos el código original y si ejecutásemos el programa con la nueva instrucción. *(0.4 puntos)*

Número instrucciones código original: ____ Número instrucciones con la nueva instrucción: ____

Ciclos de ejecución código original: ____ Ciclos de ejecución con la nueva instrucción: ____